

–Permisos y roles

/* 1. Roles-grupo (NOLOGIN) */

```
CREATE ROLE emp NOLOGIN;           -- Perfil de empleado
CREATE ROLE admin NOLOGIN IN ROLE emp; -- Perfil de admin
```

/* 2. Rol dueño (NOLOGIN) */

```
CREATE ROLE video NOLOGIN;
```

/* 3. Usuarios con sus respectivas contraseñas */

```
CREATE ROLE empleado1 LOGIN PASSWORD '12345';
CREATE ROLE administrador1 LOGIN PASSWORD '54321';
```

```
GRANT emp TO empleado1;
GRANT admin TO administrador1;
```

/* _____
4. Permisos sobre funciones y procedimientos
_____ */

```
ALTER FUNCTION public.fn_insert_customer (  
    INT, TEXT, TEXT, TEXT, TEXT, TEXT, TEXT, INT, TEXT, TEXT  
) OWNER TO video;  
ALTER FUNCTION public.fn_insert_customer (  
    INT, TEXT, TEXT, TEXT, TEXT, TEXT, TEXT, INT, TEXT, TEXT  
) SECURITY DEFINER;
```

```
ALTER FUNCTION public.fn_register_rental (INT, INT, INT)  
    OWNER TO video;  
ALTER FUNCTION public.fn_register_rental (INT, INT, INT)  
    SECURITY DEFINER;
```

```
ALTER PROCEDURE public.sp_register_return (INT, INT, NUMERIC)  
    OWNER TO video;  
ALTER PROCEDURE public.sp_register_return (INT, INT, NUMERIC)  
    SECURITY DEFINER;
```

```
ALTER FUNCTION public.fn_search_film (TEXT)  
    OWNER TO video;  
ALTER FUNCTION public.fn_search_film (TEXT)  
    SECURITY DEFINER;
```

-- Dar permiso de solo ejecución

```
GRANT EXECUTE ON FUNCTION public.fn_register_rental (INT, INT, INT) TO emp;
```

```
GRANT EXECUTE ON PROCEDURE public.sp_register_return (INT, INT, NUMERIC) TO emp;
GRANT EXECUTE ON FUNCTION public.fn_search_film (TEXT) TO emp;
```

```
-- Dar permiso al rol admin de ejecución
```

```
GRANT EXECUTE ON FUNCTION public.fn_insert_customer (
    INT, TEXT, TEXT, TEXT, TEXT, TEXT, TEXT, INT, TEXT, TEXT
) TO admin;
```

```
-- Remover los permisos que postgresql brinda por defecto
```

```
REVOKE EXECUTE ON FUNCTION public.fn_insert_customer (
    INT, TEXT, TEXT, TEXT, TEXT, TEXT, TEXT, INT, TEXT, TEXT
) FROM PUBLIC;
```

```
REVOKE EXECUTE ON FUNCTION public.fn_register_rental (INT, INT, INT) FROM PUBLIC;
```

```
REVOKE EXECUTE ON PROCEDURE public.sp_register_return (INT, INT, NUMERIC) FROM
PUBLIC;
```

```
REVOKE EXECUTE ON FUNCTION public.fn_search_film (TEXT) FROM PUBLIC;
```

```
ALTER DEFAULT PRIVILEGES FOR ROLE video
```

```
    REVOKE ALL ON TABLES FROM emp, admin;
```

```
ALTER DEFAULT PRIVILEGES FOR ROLE video
```

```
    REVOKE ALL ON SEQUENCES FROM emp, admin;
```

```
ALTER DEFAULT PRIVILEGES FOR ROLE video
```

```
    REVOKE ALL ON ROUTINES FROM emp, admin;
```

-Funciones

```
/*-----
```

Nombre : fn_insert_customer

Descripción : Inserta un nuevo registro en address y customer.

Devuelve el ID del cliente creado.

Parámetros :

p_store_id INT -> Tienda a la que pertenece el cliente.

p_first_name TEXT -> Nombre.

p_last_name TEXT -> Apellido.

p_email TEXT -> E-mail (puede ser NULL).

p_address TEXT -> Línea 1 de dirección.

p_address2 TEXT -> Línea 2 (NULL si no aplica).

p_district TEXT -> Provincia/barrio.

p_city_id INT -> FK a city.

p_postal_code TEXT -> Código postal (NULL si no aplica).

p_phone TEXT -> Teléfono.

Retorna : INT (customer_id generado).

Notas :

- Crea primero la fila en address y usa su PK para customer.
- Maneja todo en un único bloque atómico.

-----*/

```
CREATE OR REPLACE FUNCTION public.fn_insert_customer (  
  p_store_id INT,  
  p_first_name TEXT,  
  p_last_name TEXT,  
  p_email TEXT,  
  p_address TEXT,  
  p_address2 TEXT,  
  p_district TEXT,  
  p_city_id INT,  
  p_postal_code TEXT,  
  p_phone TEXT  
) RETURNS INT  
LANGUAGE plpgsql  
AS $$  
DECLARE  
  v_address_id INT;  
  v_customer_id INT;  
BEGIN  
  -- 1. Inserta la dirección  
  INSERT INTO public.address (  
    address, address2, district,  
    city_id, postal_code, phone, last_update)  
  VALUES (  
    p_address, p_address2, p_district,  
    p_city_id, p_postal_code, p_phone, now())  
  RETURNING address_id INTO v_address_id;  
  
  -- 2. Inserta el cliente  
  INSERT INTO public.customer (  
    store_id, first_name, last_name, email,  
    address_id, activebool, create_date, last_update, active)  
  VALUES (  
    p_store_id, p_first_name, p_last_name, p_email,  
    v_address_id, TRUE, now(), now(), 1)  
  RETURNING customer_id INTO v_customer_id;  
  
  RETURN v_customer_id;  
EXCEPTION  
  WHEN others THEN
```

```

        RAISE NOTICE 'Error al insertar cliente: %', SQLERRM;
    RAISE;
END;
$$;

/*-----
Nombre      : fn_register_rental
Descripción : Registra un alquiler si el inventario está disponible.
Parámetros  :
    p_customer_id INT -> Cliente que alquila.
    p_inventory_id INT -> Copia/stock a alquilar.
    p_staff_id    INT -> Empleado que atiende.
Retorna     : INT (rental_id generado).
Notas       :
    – Verifica que el inventario no tenga un rental abierto
      (return_date IS NULL).
    – Lanza excepción si la copia está ocupada.
-----*/
CREATE OR REPLACE FUNCTION public.fn_register_rental (
    p_customer_id INT,
    p_inventory_id INT,
    p_staff_id    INT
) RETURNS INT
LANGUAGE plpgsql
AS $$
DECLARE
    v_rental_id INT;
    v_busy      INT;
BEGIN

    SELECT COUNT(*) INTO v_busy
    FROM public.rental
    WHERE inventory_id = p_inventory_id
      AND return_date IS NULL;

    IF v_busy > 0 THEN
        RAISE EXCEPTION 'La copia % ya está alquilada.', p_inventory_id;
    END IF;

    INSERT INTO public.rental (
        rental_date, inventory_id, customer_id,
        staff_id, last_update)
    VALUES (now(), p_inventory_id, p_customer_id,

```

```

        p_staff_id, now())
RETURNING rental_id INTO v_rental_id;

RETURN v_rental_id;
END;
$$;

/*-----
Nombre      : sp_register_return
Descripción : Cierra un alquiler (return_date) y, opcionalmente,
              genera un pago.
Parámetros :
    p_rental_id   INT    -> Rental a cerrar.
    p_staff_id    INT    -> Empleado que recibe.
    p_payment_amt NUMERIC -> Monto pagado (0 = sin pago).
Salida      : n/a (procedure)
Notas       :
    - Valida que el alquiler exista y no esté devuelto.
    - Si p_payment_amt > 0 crea un registro en payment.
-----*/
CREATE OR REPLACE PROCEDURE public.sp_register_return (
    IN p_rental_id INT,
    IN p_staff_id INT,
    IN p_payment_amt NUMERIC DEFAULT 0
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_customer_id INT;
BEGIN
    SELECT customer_id INTO v_customer_id
    FROM public.rental
    WHERE rental_id = p_rental_id
       AND return_date IS NULL
    FOR UPDATE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Alquiler % no existe o ya fue devuelto.', p_rental_id;
    END IF;

    UPDATE public.rental
    SET return_date = now(),
        last_update = now(),
        staff_id    = p_staff_id

```

```

WHERE rental_id = p_rental_id;

IF p_payment_amt > 0 THEN
    INSERT INTO public.payment (
        customer_id, staff_id, rental_id,
        amount, payment_date)
    VALUES (
        v_customer_id, p_staff_id, p_rental_id,
        p_payment_amt, now());
END IF;
END;
$$;

/*-----
Nombre      : fn_search_film
Descripción : Devuelve lista de películas que coinciden
              (título LIKE %término%).
Parámetros  :
    p_term TEXT -> Término a buscar (case-insensitive).
Retorna     : SETOF RECORD (film_id, title, release_year, rating, length)
-----*/
CREATE OR REPLACE FUNCTION public.fn_search_film (
    p_term TEXT
)
RETURNS TABLE (
    film_id INT,
    title TEXT,
    release_year INT,
    rating MPAA_RATING,
    length INT
)
LANGUAGE sql
AS $$
    SELECT f.film_id,
           f.title,
           f.release_year,
           f.rating,
           f.length
    FROM   public.film f
    WHERE  lower(f.title) LIKE '%' || lower(p_term) || '%'
    ORDER BY f.title;
$$;

```

-- Ejemplo de uso

-- 1) Insertar cliente

```
SELECT public.fn_insert_customer(  
    1, 'Ana', 'Gómez', 'ana@example.com',  
    'Calle 1', NULL, 'San José', 300, '10101', '2222-3333');
```

-- 2) Registrar alquiler

```
SELECT public.fn_register_rental(600, 1523, 1);
```

-- 3) Registrar devolución

```
CALL public.sp_register_return(11496, 1, 3.99);
```

-- 4) Buscar película

```
SELECT * FROM public.fn_search_film('academy');
```